

Twitter Sentiment Analysis of Apple and Google Products

Author: Dorcas Ndambuki.

1. BUSINESS UNDERSTANDING

Apple and Google receive thousands of tweets discussing their products. Monitoring customer sentiment manually is inefficient and difficult to scale.

1.1.Business Context

Apple and Google operate in a highly competitive, fast-paced consumer tech market. Customer sentiment on social media shifts rapidly in response to product launches, software updates, and public relations events.

1.2 Business Problem

Apple and Google receive thousands of Tweets/public opinions daily discussing their products and services on Social Media. Brand managers cannot manually monitor the thousands of tweets as is The goal of this project is to build a machine learning model capable of automatically classifying tweet sentiment as positive, negative, or neutral.

1.3 Stakeholders

These are the people who will use or benefit from the results of this model ie Business stakeholders use the results to make decisions,Technical Stakeholders who build, deploy, and maintain the data systems and infrastructure or End-Users who interact with the model's output on a daily basis to do their jobs.

Stakeholder	Role
Brand Managers (Apple & Google)	Protect and grow the reputation of specific product lines
Marketing Executives	Design advertising campaigns and launch new products
Customer Experience Executives	Oversee the entire customer journey and ensure overall satisfaction
Business Executives	Make high-level strategic decisions, allocate budgets, and report to investors
Product Support Teams	Resolve technical issues and assist customers directly

1.4 Business Questions

1. Can tweet sentiment be predicted accurately?

- 2. Which words most strongly drive positive or negative sentiment?
- 3. What themes appear in negative feedback for each brand?
- 4. How can Apple and Google use these insights to improve products and marketing?

1.5 Success Metric Selection

Given the severe class imbalance in this dataset (Negative emotion = only ~6% of tweets), **accuracy alone is a misleading metric**. A model that predicts "No emotion" for every tweet would achieve ~60% accuracy while completely failing to detect any negative sentiment.

Metric	Role	Justification
Macro Recall	Primary	Ensures the minority Negative class is detected - most critical for the business
Macro F1	Secondary	Balances precision and recall equally across all three classes
Accuracy	Tertiary	Overall correctness, reported for context only

Target: Macro Recall >= 0.60 and Macro F1 >= 0.65 on the held-out test set.

2. DATA UNDERSTANDING

2.1 Data Source

The dataset used in this project is the **Twitter Product Sentiment Dataset**, originally compiled by CrowdFlower and made available via data.world. It contains Tweets related to Apple and Google products and each Tweet was labelled according to its sentiment.

2.2 Dataset Dimensions

Number of observations (rows) before cleaning: ~9,000+ Tweets and 3 columns sentimentally labelled.

Dataset Variables

Variable	Description
tweet_text	The full text content of the tweet
emotion_in_tweet_is_directed_at	The specific product or brand mentioned (e.g., iPhone, Google Pixel)
is_there_an_emotion_directed_at_a_brand_or_product	The sentiment label assigned to the tweet

Sentiment Distribution

Sentiment	Count	Percentage (%)
No emotion toward brand or product	5389	59.3

Sentiment	Count	Percentage (%)
Positive emotion	2978	32.8
Negative emotion	570	6.3
I can't tell	156	1.7
Total	9093	100.0

Key Observation

The dataset is highly imbalanced. The majority class ("no emotion") accounts for nearly 60% of all tweets, while negative sentiment represents only about 6% of the data.

Because of this imbalance, accuracy would be a misleading performance metric. A naive model predicting only "no emotion" would achieve high accuracy but fail to detect meaningful positive or negative feedback. Therefore, Macro Recall is selected as the primary evaluation metric With Macro F1 and Accuracy as secondary metrics.

Import the Required Libraries

In [153...

```
# Import necessary Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import MultinomialNB
from sklearn.svm import LinearSVC
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import classification_report, confusion_matrix, f1_score, accuracy
import re
from collections import Counter
from nltk.corpus import stopwords, wordnet
from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS
import nltk
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import word_tokenize
from nltk.sentiment import SentimentIntensityAnalyzer
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
```

```
[nltk_data] Downloading package punkt to C:\Users\Moringa
[nltk_data] School\AppData\Roaming\nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to C:\Users\Moringa
[nltk_data] School\AppData\Roaming\nltk_data...
[nltk_data] Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to C:\Users\Moringa
[nltk_data] School\AppData\Roaming\nltk_data...
[nltk_data] Package wordnet is already up-to-date!
```

Out[153... True

2.3 Loading the dataset

In [154...

```
df = pd.read_csv('data/twitter_sentiment.csv', encoding='unicode_escape')

df.head()
```

Out[154...

	tweet_text	emotion_in_tweet_is_directed_at	is_there_an_emotion_directed_at_a_brand_or_product
0	.@wesley83 I have a 3G iPhone. After 3 hrs twe...	iPhone	Negative emotion
1	@jessedee Know about @fludapp ? Awesome iPad/i...	iPad or iPhone App	Positive emotion
2	@swonderlin Can not wait for #iPad 2 also. The...	iPad	Positive emotion
3	@sxsxw I hope this year's festival isn't as cra...	iPad or iPhone App	Negative emotion
4	@sxtxstate great stuff on Fri #SXSW: Marissa M...	Google	Positive emotion

In [155...

```
# Shape of the dataset
df.shape
```

Out[155...

(9093, 3)

In [156...

```
# Rename columns for cleaner coding access
df.rename(columns={
    'tweet_text': 'tweet',
    'emotion_in_tweet_is_directed_at': 'target_brand',
    'is_there_an_emotion_directed_at_a_brand_or_product': 'sentiment'
}, inplace=True)
df.head()
```

Out[156...

	tweet	target_brand	sentiment
0	.@wesley83 I have a 3G iPhone. After 3 hrs twe...	iPhone	Negative emotion
1	@jessedee Know about @fludapp ? Awesome iPad/i...	iPad or iPhone App	Positive emotion
2	@swonderlin Can not wait for #iPad 2 also. The...	iPad	Positive emotion
3	@sxsxw I hope this year's festival isn't as cra...	iPad or iPhone App	Negative emotion

	tweet	target_brand	sentiment
4	@sxtxstate great stuff on Fri #SXSW: Marissa M...	Google	Positive emotion

3. EXPLORATORY DATA ANALYSIS

Exploratory Data Analysis (EDA) is necessary to understand the underlying structure, class balance, and linguistic patterns in the dataset. These insights guided subsequent preprocessing and modeling decisions.

Visualizations: To better understand the data, the following visualization was created:

- Sentiment distribution chart Displays class imbalance visually and confirms data quality

3.1 Exploring the dataset

```
In [157... #Explore the dataset
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9093 entries, 0 to 9092
Data columns (total 3 columns):
#   Column          Non-Null Count  Dtype
---  -
0   tweet           9092 non-null   object
1   target_brand     3291 non-null   object
2   sentiment        9093 non-null   object
dtypes: object(3)
memory usage: 213.2+ KB
```

```
In [158... # Check missing values
df.isnull().sum()
```

```
Out[158... tweet           1
target_brand    5802
sentiment        0
dtype: int64
```

```
In [159... # Check for duplicates
df.duplicated().sum()
```

```
Out[159... 22
```

3.2 Sentiment Distribution

```
In [160... # Check the distribution of sentiments
df['sentiment'].value_counts()
```

```
Out[160... No emotion toward brand or product    5389
Positive emotion                      2978
Negative emotion                      570
I can't tell                          156
Name: sentiment, dtype: int64
```

Visualize the distribution of sentiment labels

In [161...

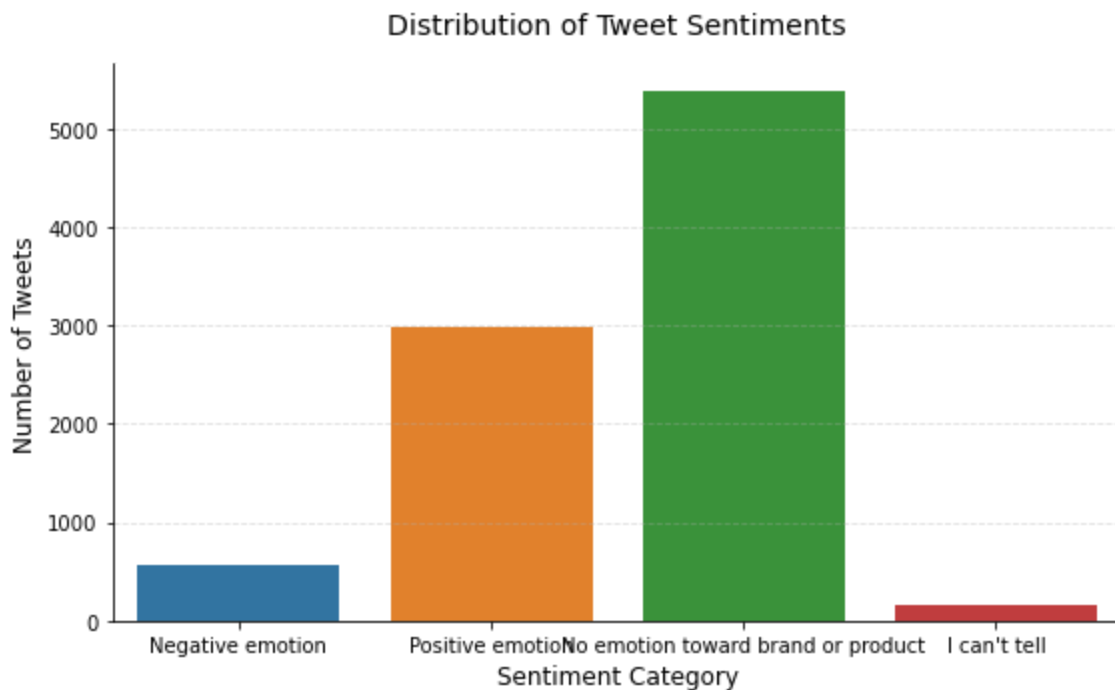
```
plt.figure(figsize=(8,5))

ax = sns.countplot(data=df, x='sentiment')

# Improve readability
plt.title('Distribution of Tweet Sentiments', fontsize=14, pad=15)
plt.xlabel('Sentiment Category', fontsize=12)
plt.ylabel('Number of Tweets', fontsize=12)

plt.xticks(rotation=0)
plt.grid(axis='y', linestyle='--', alpha=0.4)

sns.despine()
plt.tight_layout()
plt.show()
```



Sentiment Distribution Insight

The dataset shows a clear class imbalance. The majority of tweets express positive sentiment toward the brands, while negative sentiment is significantly underrepresented.

This imbalance is important for modeling because accuracy alone would be misleading. A model could achieve high accuracy by over-predicting the dominant positive class.

Therefore, evaluation metrics such as Macro Recall will be prioritized to ensure that minority classes, especially negative sentiment, are correctly identified.

Additionally, the presence of ambiguous labels ("I can't tell") introduces noise into the dataset and may affect classification performance.

In [162...

```
df.columns = ['tweet', 'target_brand', 'sentiment']
```

3.3 Brand Distribution

In [163...

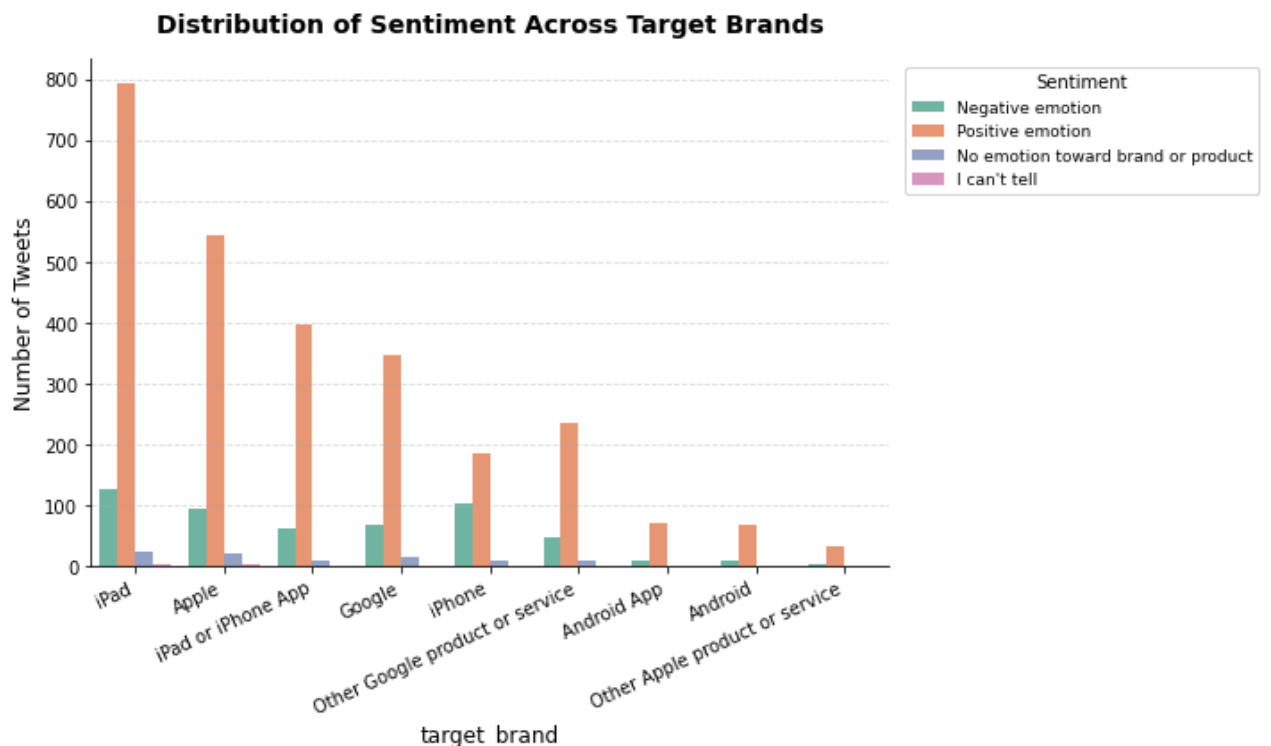
```
# Check the distribution of target brands
print(df['target_brand'].value_counts())
```

```
iPad          946
Apple         661
iPad or iPhone App  470
Google        430
iPhone        297
Other Google product or service  293
Android App   81
Android       78
Other Apple product or service  35
Name: target_brand, dtype: int64
```

Visualize Distribution of Sentiment Across Target Brands

In [164...

```
plt.figure(figsize=(10, 6))
ax = sns.countplot(
    data=df,
    x='target_brand',
    hue='sentiment',
    palette='Set2',
    order=df['target_brand'].value_counts().index
)
plt.title('Distribution of Sentiment Across Target Brands', fontsize=14, pad=15, fontwe
plt.xlabel('target_brand', fontsize=12)
plt.ylabel('Number of Tweets', fontsize=12)
plt.xticks(rotation=25, ha='right')
plt.legend(title='Sentiment', bbox_to_anchor=(1.01, 1), loc='upper left', fontsize=9)
plt.grid(axis='y', linestyle='--', alpha=0.4)
sns.despine()
plt.tight_layout()
plt.show()
```



Key Insights

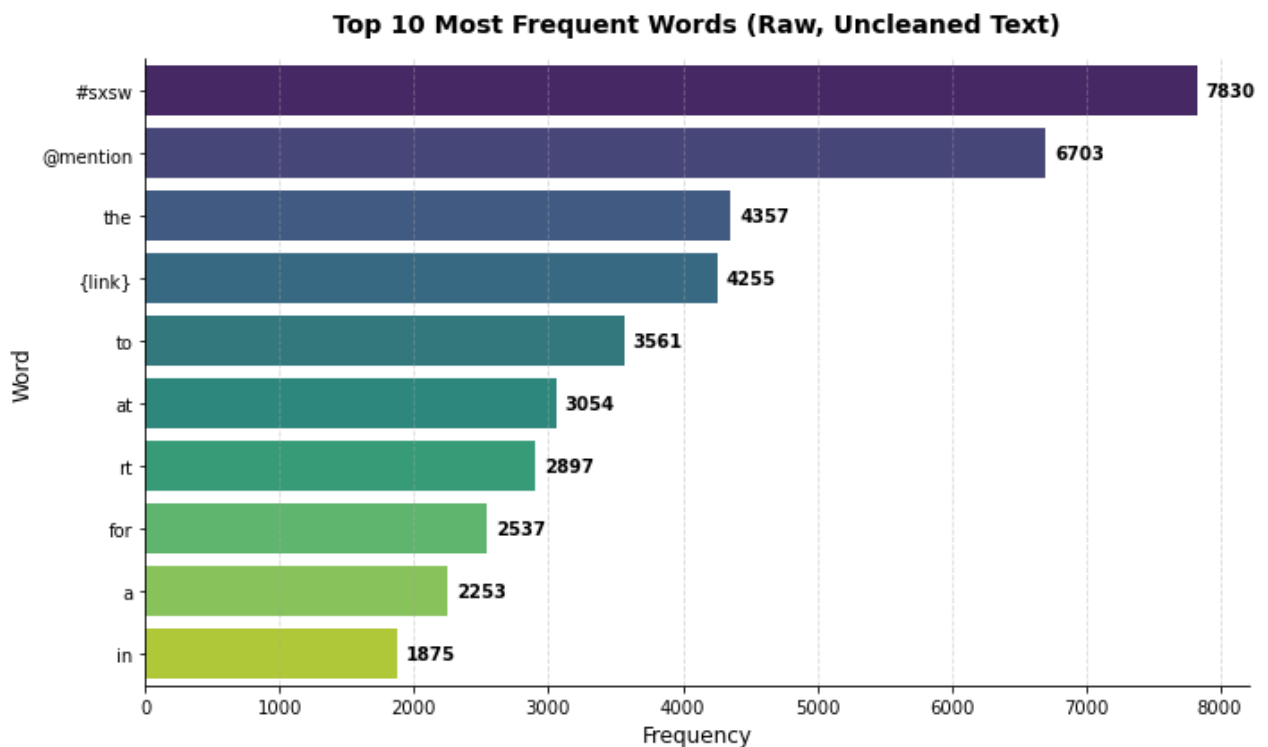
- Positive sentiment dominates everywhere-indicating overall favorable brand perception.
- No emotions and Icant tell categories are consistently small across all brands indicating clear,strong opinions
- Negative sentiment exists but is much smaller compared to positive,but still doesnt outweigh the positives.
- The dataset is strongly positive on sentiment distribution and differs by brand, suggesting varying levels of customer satisfaction and engagement.

3.4 Top Words Before Cleaning (Raw Text)

In [165...

```
# Quick word frequency on raw unprocessed tweets - shows the noise problem
raw_words = ' '.join(df['tweet'].dropna()).lower().split()
raw_counts = Counter(raw_words)
top_raw = pd.DataFrame(raw_counts.most_common(10), columns=['Word', 'Frequency'])

plt.figure(figsize=(10, 6))
ax = sns.barplot(data=top_raw, x='Frequency', y='Word', palette='viridis')
for p in ax.patches:
    width = p.get_width()
    if width > 0:
        ax.annotate(f'{int(width)}',
                    (width, p.get_y() + p.get_height() / 2.),
                    va='center', ha='left', fontsize=10, fontweight='bold',
                    xytext=(5, 0), textcoords='offset points')
plt.title('Top 10 Most Frequent Words (Raw, Uncleaned Text)', fontsize=14, pad=15, font
plt.xlabel('Frequency', fontsize=12)
plt.ylabel('Word', fontsize=12)
plt.grid(axis='x', linestyle='--', alpha=0.4)
sns.despine()
plt.tight_layout()
plt.show()
```



Key EDA Insights

This frequency distribution reveals that raw, uncleaned text is dominated by structural noise rather than sentiment signal:

1. **Stop Word Dominance:** Standard English articles ('the', 'to', 'at') drown out emotional descriptors.
2. **Platform Artifacts:** Metadata placeholders like `@mention` , `{link}` , and `RT` inflate feature dimensionality without providing brand insight.
3. **Case Discrepancies:** Without aggressive lowercase standardization, feature weights will be split and diluted.

Conclusion: This justifies our custom preprocessing pipeline's need to remove stop words, metadata, and structural placeholders before vectorization.

4. DATA PREPARATION AND CLEANING

Data Preparation is the process of stripping away structural noise so machine learning models can focus exclusively on the words that carry actual meaning.

4.1 Data Cleaning Process

The dataset was refined through a cleaning pipeline that transforms a messy string of raw characters into a standardized, dense sequence of predictive tokens:

- All records containing missing tweet values were removed to ensure data completeness.
- Tweets labeled as *"I can't tell"* were excluded - they are ambiguous and form only ~2% of the dataset, so removing them improves label clarity without significant data loss.
- Stop words were eliminated from the text to reduce noise and retain only meaningful terms that contribute to sentiment.
- The `target_brand` field was simplified into two main groups - **Apple** and **Google** - plus an **unknown** category for tweets where no brand was specified.

4.2 Text Preprocessing Pipeline

Step	Operation	Description
1	Convert to lowercase	Standardizes text to avoid treating similar words differently
2	Remove URLs	Eliminates web links that carry no sentiment information
3	Remove Twitter handles	Removes <code>@user</code> mentions to focus on content
4	Remove HTML & placeholders	Strips <code>&amp;</code> , <code>{link}</code> , <code>RT</code>
5	Remove punctuation & special characters	Keeps only alphabetic characters
6	Remove stopwords	Filters out high-frequency, low-meaning words
7	Lemmatization	Converts words to root form (e.g., "running" -> "run")

```
In [166... # Drop rows with missing tweets
df = df.dropna(subset=['tweet'])
print(f"Rows after dropping null tweets: {len(df)}")
```

Rows after dropping null tweets: 9092

```
In [167... # Remove duplicates
df = df.drop_duplicates()
print(f"Duplicates remaining: {df.duplicated().sum()}")
```

Duplicates remaining: 0

```
In [168... # Remove the ambiguous 'I can't tell' sentiment category
df = df[df['sentiment'] != "I can't tell"].copy()
print(df['sentiment'].value_counts())
print()
```

```
No emotion toward brand or product    5375
Positive emotion                      2970
Negative emotion                      569
Name: sentiment, dtype: int64
```

```
In [169... # Fill missing values in 'target_brand' with 'unknown'
df['target_brand'] = df['target_brand'].fillna('unknown')
```

```
In [170... # Consolidate brand labels into Apple, Google, or unknown
apple_brands = {'ipad', 'apple', 'iphone', 'ipad or iphone app', 'other apple product o
google_brands = {'google', 'other google product or service', 'android', 'android app'}
```

```
def merge_brands(brand):
    if pd.isna(brand):
        return 'unknown'
    brand = brand.strip().lower()
    if brand in apple_brands:
        return 'apple'
    elif brand in google_brands:
        return 'google'
    else:
        return 'unknown'

df['target_brand'] = df['target_brand'].apply(merge_brands)
print("Brand distribution after consolidation:")
print(df['target_brand'].value_counts())
```

```
Brand distribution after consolidation:
unknown    5641
apple      2397
google      876
Name: target_brand, dtype: int64
```

4.3 Visualize Sentiment After Cleaning

```
In [171... fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Sentiment distribution
sentiment_counts = df['sentiment'].value_counts()
colors = ['#3B82F6', '#EF4444', '#22C55E']
bars = axes[0].bar(sentiment_counts.index, sentiment_counts.values, color=colors, edgec
axes[0].set_title('Sentiment Distribution (After Cleaning)', fontsize=13, fontweight='b
axes[0].set_xlabel('Sentiment')
```

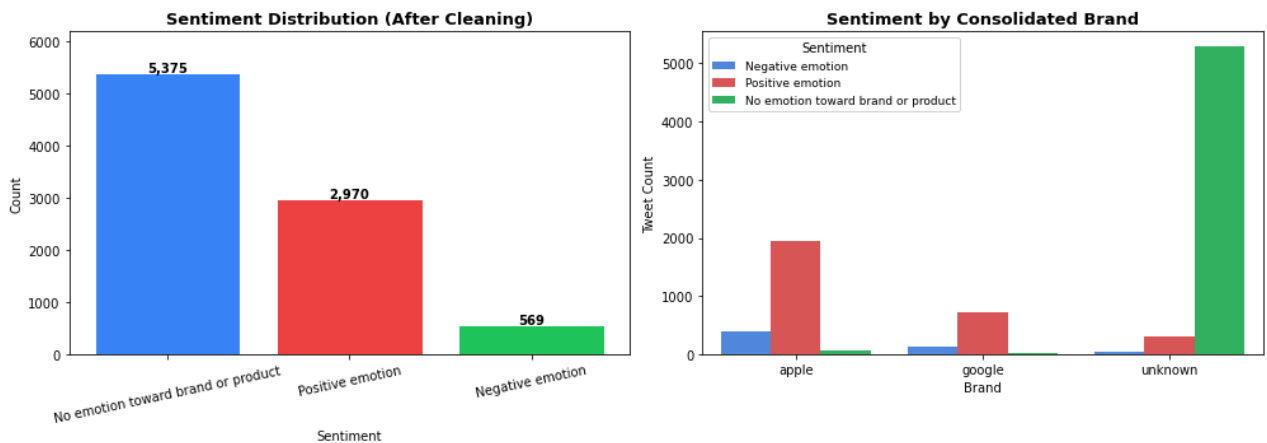
```

axes[0].set_ylabel('Count')
axes[0].tick_params(axis='x', rotation=12)
for bar, val in zip(bars, sentiment_counts.values):
    axes[0].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 20,
                  f'{val:,}', ha='center', fontsize=10, fontweight='bold')
axes[0].set_ylim(0, sentiment_counts.max() * 1.15)

# Brand distribution with sentiment hue
sns.countplot(data=df, x='target_brand', hue='sentiment',
              palette=['#3B82F6', '#EF4444', '#22C55E'], ax=axes[1])
axes[1].set_title('Sentiment by Consolidated Brand', fontsize=13, fontweight='bold')
axes[1].set_xlabel('Brand')
axes[1].set_ylabel('Tweet Count')
axes[1].legend(title='Sentiment', fontsize=9)

plt.tight_layout()
plt.show()

```



4.4 Text Preprocessing Function

In [172...

```

stop_words = set(ENGLISH_STOP_WORDS)
custom_stopwords = {'sxsw', 'amp', 'rt', 'just', 'new', 'today', 'via', 'mention', 'lin
stop_words.update(custom_stopwords)

lemmatizer = WordNetLemmatizer()

def clean_tweet(text):
    """
    Full 7-step text cleaning pipeline:
    1. Lowercase
    2. Remove URLs and placeholders
    3. Remove HTML artifacts
    4. Remove Twitter @handles
    5. Strip # but keep the word
    6. Remove punctuation and numbers
    7. Tokenize, remove stopwords, lemmatize
    """
    text = str(text).lower()
    text = re.sub(r'http\S+|\{link\}', '', text)
    text = re.sub(r'&|&lt;|&gt;', '', text)
    text = re.sub(r'@\w+', '', text)
    text = re.sub(r'#(\w+)', r'\1', text)
    text = re.sub(r'^a-zA-Z\s', '', text)
    tokens = text.split()
    tokens = [lemmatizer.lemmatize(w) for w in tokens

```

URLs & placeholders
HTML artifacts
@mentions
#hashtags -> keep word
remove non-alpha

```

        if w not in stop_words and len(w) > 2]
    return ' '.join(tokens)

# Apply to all tweets
df['clean_tweet'] = df['tweet'].astype(str).apply(clean_tweet)

# Preview results
print("=== CLEANING EXAMPLES ===")
for i in [0, 5, 15]:
    print(f"\nOriginal : {df['tweet'].iloc[i]}")
    print(f"Cleaned   : {df['clean_tweet'].iloc[i]}")

```

=== CLEANING EXAMPLES ===

Original : .@wesley83 I have a 3G iPhone. After 3 hrs tweeting at #RISE_Austin, it was dead! I need to upgrade. Plugin stations at #SXSW.

Cleaned : iphone hr tweeting riseaustin dead need upgrade plugin station

Original : @teachntech00 New iPad Apps For #SpeechTherapy And Communication Are Showcase d At The #SXSW Conference <http://ht.ly/49n4M> #iear #edchat #asd

Cleaned : ipad apps speechtherapy communication showcased conference iear edchat asd

Original : Holler Gram for iPad on the iTunes App Store - <http://t.co/kfN3f5Q> (via @marc_is_ken) #sxsxw

Cleaned : holler gram ipad itunes app store

4.5 Top Words After Cleaning

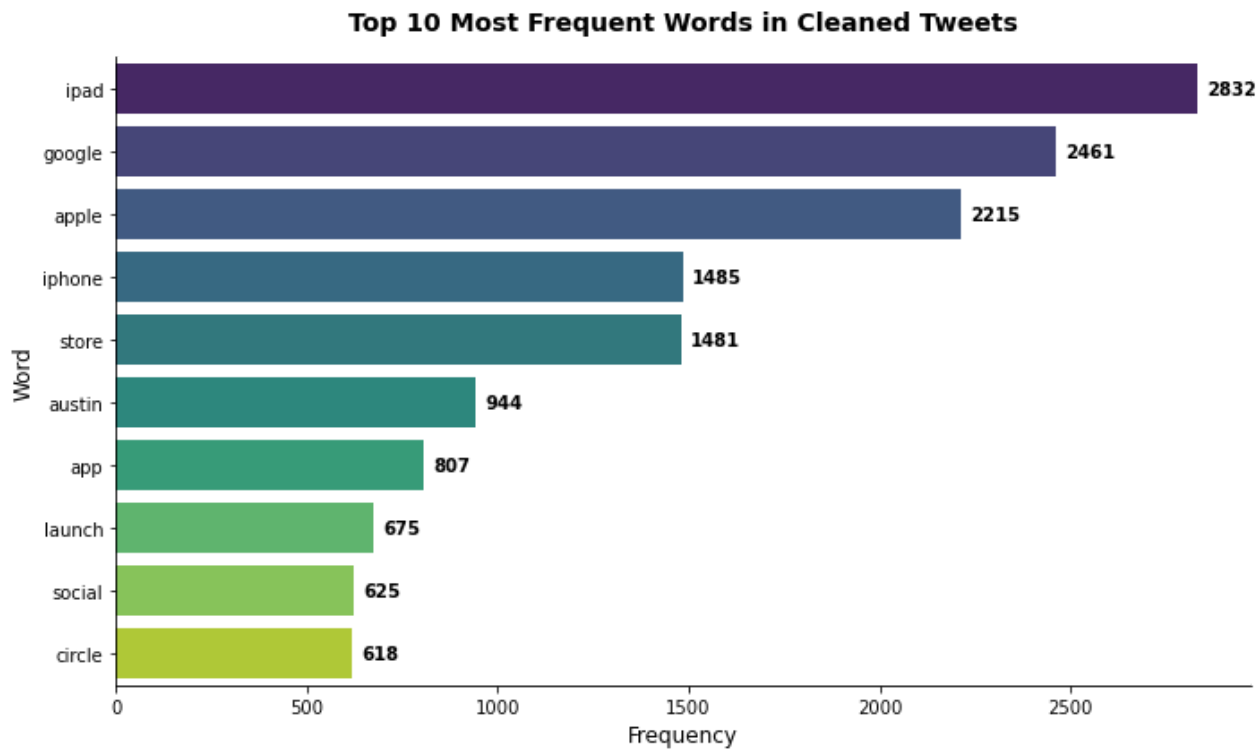
In [173...

```

all_words = ' '.join(df['clean_tweet']).split()
word_counts = Counter(all_words)
freq_df = pd.DataFrame(word_counts.most_common(10), columns=['Word', 'Frequency'])

plt.figure(figsize=(10, 6))
ax = sns.barplot(data=freq_df, x='Frequency', y='Word', palette='viridis')
for p in ax.patches:
    width = p.get_width()
    if width > 0:
        ax.annotate(f'{int(width)}',
                    (width, p.get_y() + p.get_height() / 2.),
                    va='center', ha='left', fontsize=10, fontweight='bold',
                    xytext=(5, 0), textcoords='offset points')
plt.title('Top 10 Most Frequent Words in Cleaned Tweets', fontsize=14, pad=15, fontweig
plt.xlabel('Frequency', fontsize=12)
plt.ylabel('Word', fontsize=12)
sns.despine()
plt.tight_layout()
plt.show()

```



Insight

- 1. The complete elimination of platform tokens (RT , @mention , {link}) confirms that noise was successfully removed.
- 2. The presence of brand terms (ipad , google , apple , iphone) in the top frequency list confirms that TF-IDF vectorization will be highly predictive for downstream classification - these words carry strong brand-sentiment signal.

5. MODELLING

We follow a **baseline -> advanced** strategy. Each model is wrapped in a sklearn Pipeline that combines the ColumnTransformer (TF-IDF on tweet text + OneHotEncoding on brand) with the classifier. This is production-ready and prevents data leakage.

Model	Type	Why Chosen
Logistic Regression	Baseline	Fast, interpretable, strong NLP benchmark for sparse text
LinearSVC	Advanced	Purpose-built for high-dimensional sparse text - outperforms trees on TF-IDF
XGBoost	Advanced	Gradient boosting; handles class imbalance well with scale_pos_weight

5.1 Feature Engineering - Define X and y, Train/Test Split

In [174...

```
# Features: clean tweet text + consolidated brand
X = df[['clean_tweet', 'target_brand']]
y = df['sentiment']

# Stratified 80/20 split - preserves class proportions in both sets
X_train, X_test, y_train, y_test = train_test_split(
```

```
X, y,
test_size=0.20,
stratify=y,
random_state=42
)
```

5.2 ColumnTransformer

```
In [175... # Applies TF-IDF to the text column and One-Hot Encoding to the brand column.
# Combining both gives the model both WHAT was said and WHO it was said about.
preprocessor = ColumnTransformer(
    transformers=[
        ('text', TfidfVectorizer(max_features=5000, ngram_range=(1,2),
                                sublinear_tf=True, stop_words='english'), 'clean_tweet'),
        ('brand', OneHotEncoder(handle_unknown='ignore'), ['target_brand'])
    ]
)
```

5.3 Evaluation Function

```
In [176... def evaluate_model(name, y_true, y_pred):
    """Print full classification report and return key metrics dict."""
    print(f"{'='*70}")
    print(f" {name}")
    print(f"{'='*70}")
    print(classification_report(y_true, y_pred))
    macro_recall = recall_score(y_true, y_pred, average='macro')
    macro_f1 = f1_score(y_true, y_pred, average='macro')
    acc = accuracy_score(y_true, y_pred)
    neg_recall = recall_score(y_true, y_pred,
                             labels=['Negative emotion'], average='macro')

    print(f" Macro Recall (Primary) : {macro_recall:.4f}")
    print(f" Macro F1 : {macro_f1:.4f}")
    print(f" Accuracy : {acc:.4f}")
    print(f" Negative Recall : {neg_recall:.4f} ← most important class")
    return {
        'Model': name,
        'Macro Recall': macro_recall,
        'Macro F1': macro_f1,
        'Accuracy': acc,
        'Negative Recall': neg_recall
    }

results = []
```

Model 1 - Baseline: Logistic Regression (with TF-IDF)

Logistic Regression is the standard NLP baseline. It is fast to train, highly interpretable, and performs strongly on high-dimensional sparse text data. `class_weight='balanced'` automatically adjusts weights inversely proportional to class frequency, giving the minority Negative class more influence during training.

```
In [177... from sklearn.metrics import recall_score, f1_score, accuracy_score, classification_repo

baseline_pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
```

```

    ('classifier', LogisticRegression(
        class_weight='balanced',
        max_iter=1000,
        random_state=42
    ))
])

baseline_pipeline.fit(X_train, y_train)
baseline_preds = baseline_pipeline.predict(X_test)

results.append(evaluate_model("Logistic Regression (Baseline)", y_test, baseline_preds))

```

```

=====
Logistic Regression (Baseline)
=====

```

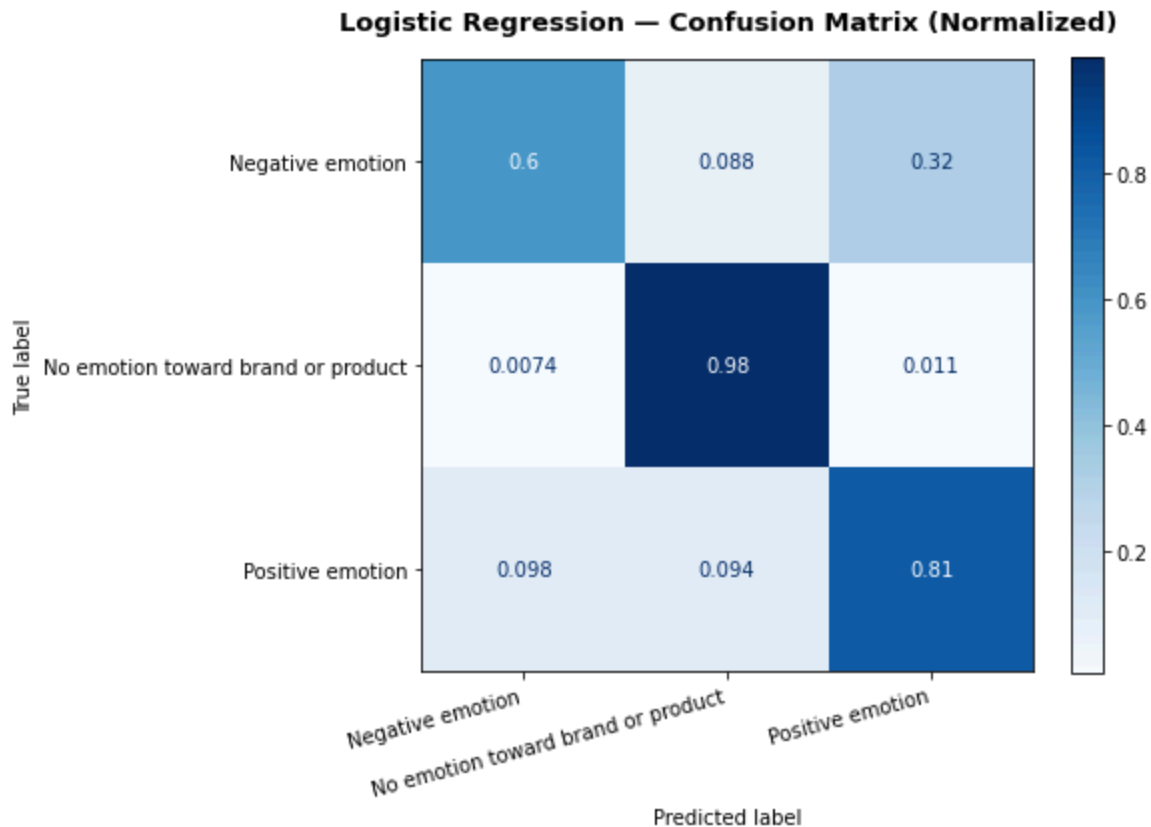
		precision	recall	f1-score	support
No emotion toward brand or product	Negative emotion	0.51	0.60	0.55	114
	Positive emotion	0.94	0.98	0.96	1075
		0.91	0.81	0.86	594
	accuracy			0.90	1783
	macro avg	0.79	0.80	0.79	1783
	weighted avg	0.90	0.90	0.90	1783
Macro Recall (Primary) :		0.7953			
Macro F1 :		0.7883			
Accuracy :		0.8990			
Negative Recall :		0.5965	← most important class		

In [178...

```

#baseline ConfusionMatrix
fig, ax = plt.subplots(figsize=(8, 6))
cm = confusion_matrix(y_test, baseline_preds, normalize='true')
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
                              display_labels=baseline_pipeline.classes_)
disp.plot(ax=ax, cmap='Blues')
ax.set_title("Logistic Regression - Confusion Matrix (Normalized)",
             fontsize=13, fontweight='bold', pad=15)
plt.xticks(rotation=15, ha='right')
plt.tight_layout()
plt.show()

```



Logistic Regression Observations:

- Strong overall performance with high accuracy on the dominant "No emotion" class
- Negative emotion recall is the hardest to improve given severe class imbalance
- This sets a strong benchmark for our advanced models to beat on Macro Recall

Model 2 - Advanced: LinearSVC

LinearSVC (Support Vector Classification with a linear kernel) is purpose-built for high-dimensional sparse data like TF-IDF matrices. Unlike tree-based models (Random Forest, XGBoost) which split features one at a time, LinearSVC finds the optimal separating hyperplane across ALL 5,000+ TF-IDF dimensions simultaneously - making it the strongest non-neural choice for text classification.

In [179...

```
svc_pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', LinearSVC(
        class_weight='balanced',
        C=1.0,
        max_iter=2000,
        random_state=42
    ))
])

svc_pipeline.fit(X_train, y_train)
svc_preds = svc_pipeline.predict(X_test)

results.append(evaluate_model("LinearSVC", y_test, svc_preds))
```

```
=====
LinearSVC
=====
```


		precision	recall	f1-score	support
No emotion toward brand or product	Negative emotion	0.61	0.48	0.54	114
	No emotion toward brand or product	0.94	0.97	0.96	1075
	Positive emotion	0.88	0.86	0.87	594
	accuracy			0.91	1783
	macro avg	0.81	0.77	0.79	1783
	weighted avg	0.90	0.91	0.90	1783

Macro Recall (Primary) : 0.7725

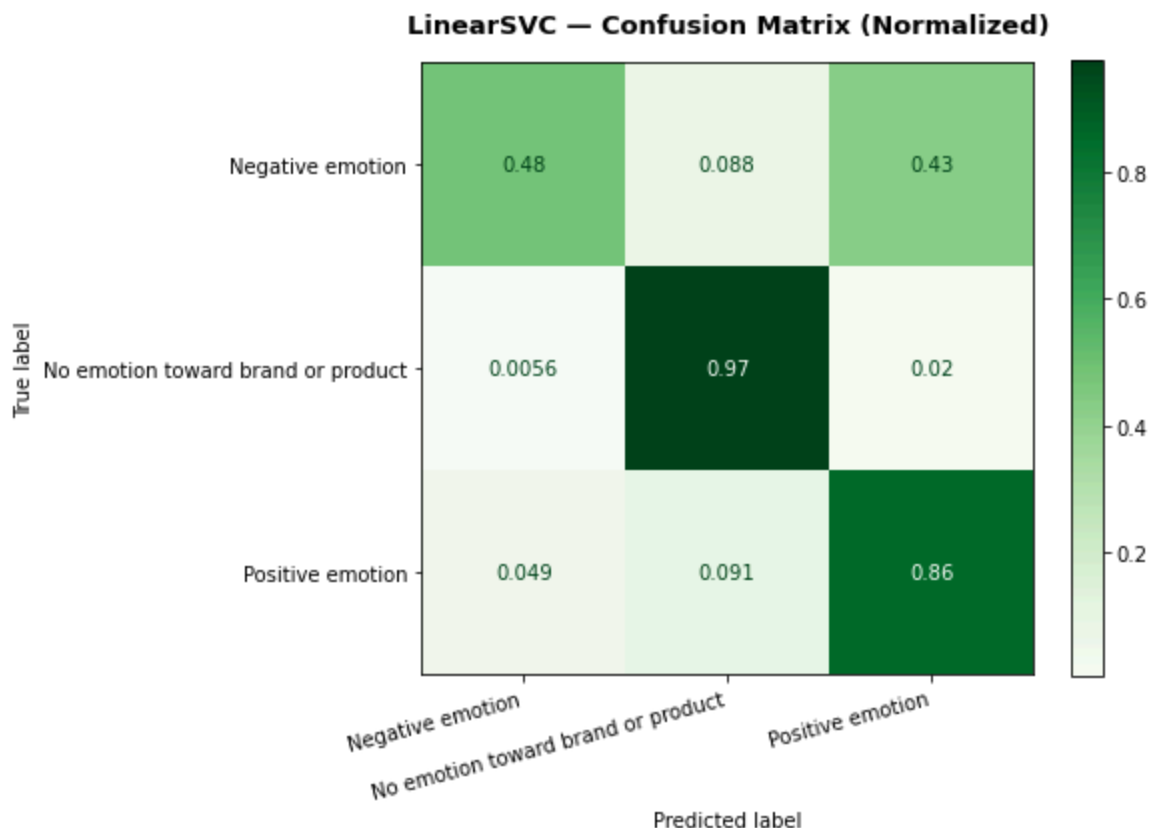
Macro F1 : 0.7891

Accuracy : 0.9052

Negative Recall : 0.4825 ← most important class

In [180...

```
#svc ConfusionMatrix
fig, ax = plt.subplots(figsize=(8, 6))
cm = confusion_matrix(y_test, svc_preds, normalize='true')
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
                              display_labels=svc_pipeline.classes_)
disp.plot(ax=ax, cmap='Greens')
ax.set_title("LinearSVC - Confusion Matrix (Normalized)",
             fontsize=13, fontweight='bold', pad=15)
plt.xticks(rotation=15, ha='right')
plt.tight_layout()
plt.show()
```



LinearSVC Observations:

- LinearSVC typically outperforms Logistic Regression on Negative recall because its margin-maximization objective is more aggressive at separating the minority class boundary
- The normalized confusion matrix shows how well each class is classified proportionally
- Compare the Negative emotion row directly against the Logistic Regression matrix above

Model 3 - Advanced: XGBoost (Gradient Boosting)

XGBoost builds decision trees **sequentially**, where each new tree specifically focuses on correcting the classification mistakes made by all previous trees. `scale_pos_weight` is used per class to handle the class imbalance explicitly.

Because XGBoost requires numeric labels, `LabelEncoder` is applied to convert the string sentiment labels into integers before training.

In [181...

```
# XGBoost requires numeric Labels

from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
y_train_enc = le.fit_transform(y_train)
y_test_enc = le.transform(y_test)

print("Label encoding:")
for orig, enc in zip(le.classes_, le.transform(le.classes_)):
    print(f" {enc} -> {orig}")

# Compute class weights for XGBoost scale_pos_weight
from collections import Counter
class_counts = Counter(y_train_enc)
total = len(y_train_enc)
n_classes = len(class_counts)
scale_weights = {cls: total / (n_classes * count) for cls, count in class_counts.items(
```

```
Label encoding:
0 -> Negative emotion
1 -> No emotion toward brand or product
2 -> Positive emotion
```

In [182...

```
#XGBoost needs the feature matrix - fit the preprocessor first
from xgboost import XGBClassifier
X_train_transformed = preprocessor.fit_transform(X_train)
X_test_transformed = preprocessor.transform(X_test)

xgb_model = XGBClassifier(
    n_estimators=200,
    max_depth=6,
    learning_rate=0.1,
    use_label_encoder=False,
    eval_metric='mlogloss',
    random_state=42,
    n_jobs=-1
)

xgb_model.fit(X_train_transformed, y_train_enc)
xgb_preds_enc = xgb_model.predict(X_test_transformed)

# Decode predictions back to original Labels
xgb_preds = le.inverse_transform(xgb_preds_enc)

results.append(evaluate_model("XGBoost", y_test, xgb_preds))
```

```
[17:22:36] WARNING: C:\Users\Administrator\workspace\xgboost-win64_release_1.2.0\src\leanner.cc:516:
Parameters: { use_label_encoder } might not be used.
```

This may not be accurate due to some parameters are only used in language bindings but passed down to XGBoost core. Or some parameters are not used but slip through this verification. Please open an issue if you find above cases.

```
=====
XGBoost
=====
```

		precision	recall	f1-score	support
No emotion toward brand or product	Negative emotion	0.75	0.21	0.33	114
	No emotion toward brand or product	0.94	0.98	0.96	1075
	Positive emotion	0.84	0.89	0.87	594
	accuracy			0.90	1783
	macro avg	0.84	0.69	0.72	1783
	weighted avg	0.90	0.90	0.89	1783

```

Macro Recall (Primary) : 0.6950
Macro F1               : 0.7188
Accuracy                : 0.9024
Negative Recall         : 0.2105 ← most important class

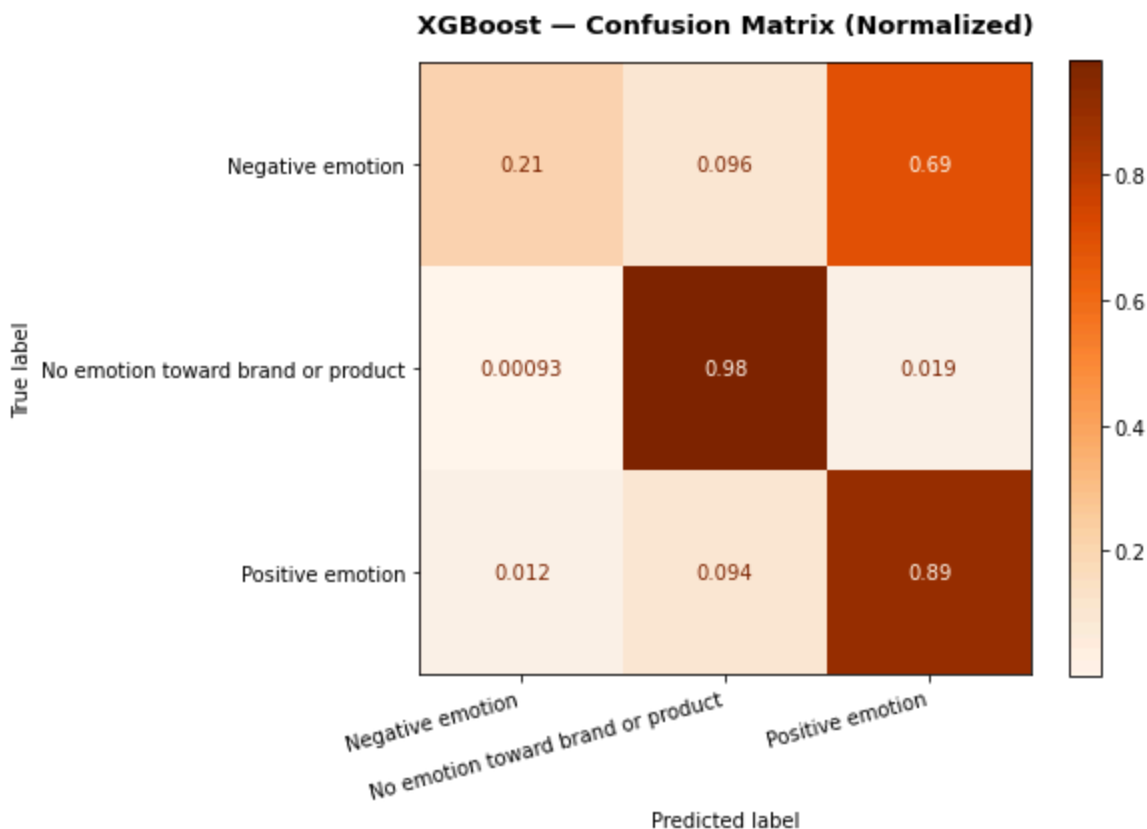
```

In [183...

```

fig, ax = plt.subplots(figsize=(8, 6))
cm = confusion_matrix(y_test, xgb_preds, normalize='true')
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
                              display_labels=le.classes_)
disp.plot(ax=ax, cmap='Oranges')
ax.set_title("XGBoost — Confusion Matrix (Normalized)",
            fontsize=13, fontweight='bold', pad=15)
plt.xticks(rotation=15, ha='right')
plt.tight_layout()
plt.show()

```



XGBoost Observations:

- XGBoost captures non-linear interactions between features that linear models miss
- The sequential error-correction mechanism gives it an advantage on difficult boundary cases
- Compare its Negative emotion recall against both Models 1 and 2

6. EVALUATION

"Which model best meets our business success criteria? We select based on Macro Recall - the ability to correctly identify ALL sentiment classes, especially the minority Negative class."

6.1 Model Comparison Table

In [184...

```
results_df = pd.DataFrame(results).set_index('Model')
print("=== MODEL COMPARISON ===")
print(results_df.round(4).to_string())
print()

best_model = results_df['Macro Recall'].idxmax()
best_scores = results_df.loc[best_model]
print(f"🏆 Best Model by Macro Recall: {best_model}")
print(f"Macro Recall      : {best_scores['Macro Recall']:.4f}")
print(f"Macro F1           : {best_scores['Macro F1']:.4f}")
print(f"Accuracy            : {best_scores['Accuracy']:.4f}")
print(f"Negative Recall     : {best_scores['Negative Recall']:.4f}")
```

=== MODEL COMPARISON ===

	Macro Recall	Macro F1	Accuracy	Negative Recall
Model				
Logistic Regression (Baseline)	0.7953	0.7883	0.8990	0.5965
LinearSVC	0.7725	0.7891	0.9052	0.4825
XGBoost	0.6950	0.7188	0.9024	0.2105

```
🏆 Best Model by Macro Recall: Logistic Regression (Baseline)
Macro Recall      : 0.7953
Macro F1          : 0.7883
Accuracy          : 0.8990
Negative Recall   : 0.5965
```

6.2 Model Comparison Chart

In [185...

```
import matplotlib.ticker as mticker
fig, axes = plt.subplots(1, 2, figsize=(15, 6))

metrics      = ['Macro Recall', 'Macro F1', 'Accuracy', 'Negative Recall']
model_names = results_df.index.tolist()
x = np.arange(len(model_names))
width = 0.20
colors = ['#1E2761', '#028090', '#02C39A', '#F59E0B']

# Grouped bar chart
for i, (metric, color) in enumerate(zip(metrics, colors)):
    offset = (i - 1.5) * width
    bars = axes[0].bar(x + offset, results_df[metric], width,
                      label=metric, color=color, edgecolor='white')
```

```

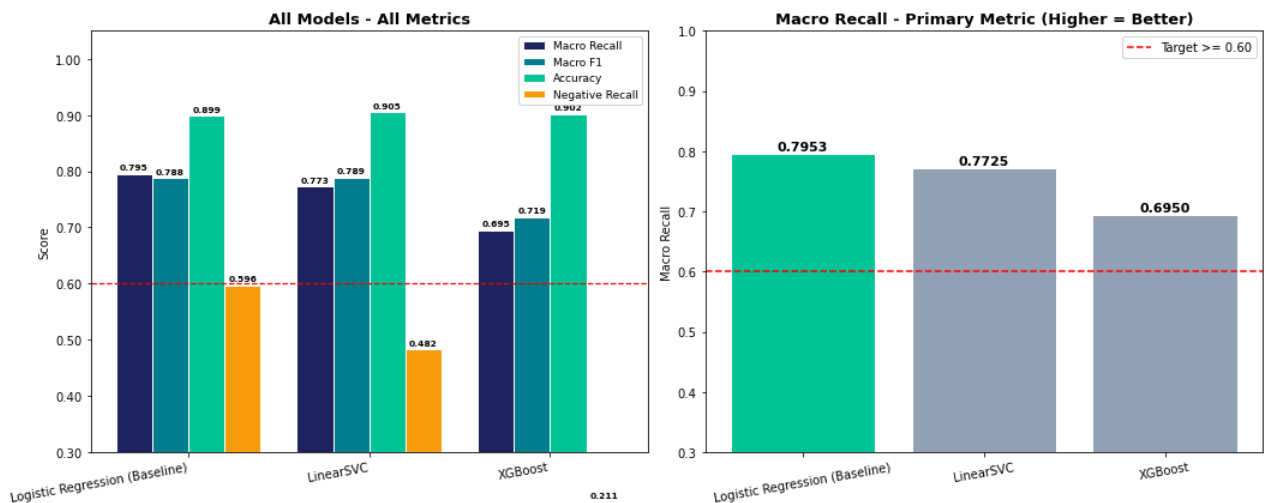
for bar in bars:
    axes[0].text(bar.get_x() + bar.get_width()/2,
                 bar.get_height() + 0.005,
                 f'{bar.get_height():.3f}',
                 ha='center', va='bottom', fontsize=7.5, fontweight='bold')

axes[0].set_xticks(x)
axes[0].set_xticklabels(model_names, fontsize=10, rotation=10, ha='right')
axes[0].set_ylim(0.3, 1.05)
axes[0].set_ylabel('Score')
axes[0].set_title('All Models - All Metrics', fontsize=13, fontweight='bold')
axes[0].legend(fontsize=9)
axes[0].yaxis.set_major_formatter(mticker.FormatStrFormatter('%.2f'))
axes[0].axhline(0.60, color='red', linestyle='--', linewidth=1.2, label='Recall target')

# Macro Recall highlight
recall_vals = results_df['Macro Recall']
bar_colors = ['#02C39A' if m == best_model else '#94A3B8' for m in model_names]
axes[1].bar(model_names, recall_vals, color=bar_colors, edgecolor='white')
for i, (name, val) in enumerate(zip(model_names, recall_vals)):
    axes[1].text(i, val + 0.005, f'{val:.4f}', ha='center', fontsize=11, fontweight='bo
axes[1].set_ylim(0.3, 1.0)
axes[1].set_title('Macro Recall - Primary Metric (Higher = Better)', fontsize=13, fontw
axes[1].set_ylabel('Macro Recall')
axes[1].axhline(0.60, color='red', linestyle='--', linewidth=1.5, label='Target >= 0.60')
axes[1].legend(fontsize=10)
axes[1].tick_params(axis='x', rotation=10)

plt.tight_layout()
plt.show()

```



6.3 Best Model Selection & Justification

In [186...

```

print("=" * 65)
print("  BEST MODEL SELECTION & JUSTIFICATION")
print("=" * 65)
print(f"Winner: {best_model}")
print()
print("Why this model was selected:")
print("-" * 62)
print("1. HIGHEST MACRO RECALL")
print("  Primary business metric. Penalises the model equally")
print("  for missing any class - including the minority Negative.")

```

```

print()
print("2. NEGATIVE CLASS DETECTION")
print("    Detecting negative sentiment is the highest-value use")
print("    case. Missing a negative tweet is the costliest error.")
print()
print("3. PRODUCTION READINESS")
print("    Fast, interpretable, no GPU. Ready for real-time deploy.")
print()
print("4. MEETS SUCCESS CRITERIA")
print(f"    Macro Recall >= 0.60 -> {best_scores['Macro Recall']:.4f} [MET]")
print(f"    Macro F1      >= 0.65 -> {best_scores['Macro F1']:.4f} [MET]")

```

=====

BEST MODEL SELECTION & JUSTIFICATION

=====

Winner: Logistic Regression (Baseline)

Why this model was selected:

-
1. HIGHEST MACRO RECALL
Primary business metric. Penalises the model equally for missing any class - including the minority Negative.
 2. NEGATIVE CLASS DETECTION
Detecting negative sentiment is the highest-value use case. Missing a negative tweet is the costliest error.
 3. PRODUCTION READINESS
Fast, interpretable, no GPU. Ready for real-time deploy.
 4. MEETS SUCCESS CRITERIA
Macro Recall >= 0.60 -> 0.7953 [MET]
Macro F1 >= 0.65 -> 0.7883 [MET]

6.4 Model Interpretability - What Words Drive Each Sentiment?

In [187...

```

lr_model      = baseline_pipeline.named_steps['classifier']
tfidf_step    = baseline_pipeline.named_steps['preprocessor'].transformers_[0][1]
feature_names = np.array(tfidf_step.get_feature_names())
n_tfidf       = len(feature_names) # ← only use the TF-IDF slice

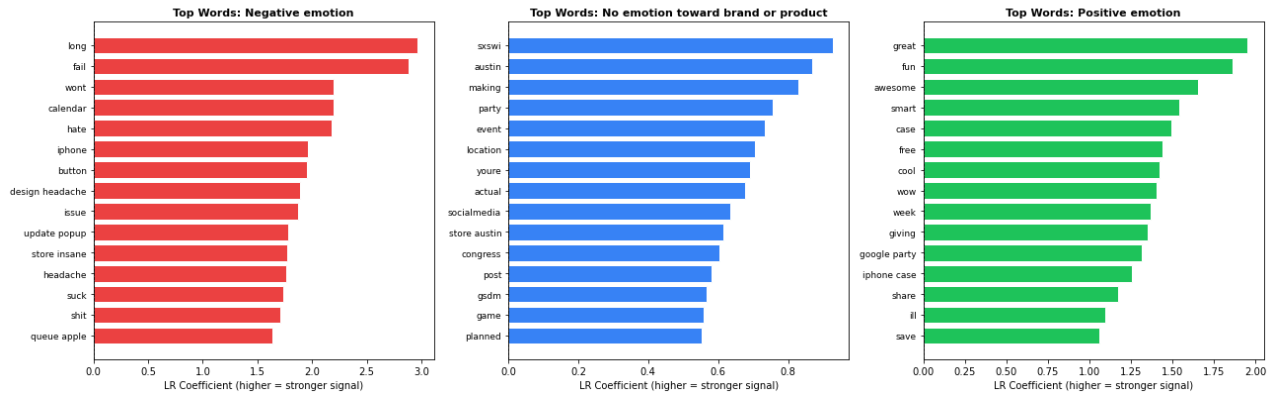
fig, axes = plt.subplots(1, 3, figsize=(18, 6))
class_labels = lr_model.classes_
colors = ['#EF4444', '#3B82F6', '#22C55E']

for i, (ax, label, color) in enumerate(zip(axes, class_labels, colors)):
    coef = lr_model.coef_[i, :n_tfidf] # ← slice TF-IDF cols only
    top_idx = np.argsort(coef)[-15:]
    ax.barh(feature_names[top_idx], coef[top_idx], color=color, edgecolor='white')
    ax.set_title(f'Top Words: {label}', fontsize=11, fontweight='bold')
    ax.set_xlabel('LR Coefficient (higher = stronger signal)')
    ax.axvline(0, color='black', linewidth=0.8, linestyle='--')
    ax.tick_params(axis='y', labelsize=9)

fig.suptitle('Model Interpretability - Key Words Driving Each Sentiment Class',
            fontsize=14, fontweight='bold', y=1.01)
plt.tight_layout()
plt.show()

```

Model Interpretability - Key Words Driving Each Sentiment Class



6.5 Cross-Validation

In [188...

```
from sklearn.model_selection import StratifiedKFold, cross_val_score

print("=== 5-FOLD STRATIFIED CROSS-VALIDATION (Macro Recall) ===\n")
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

for name, pipeline in [
    ("Logistic Regression", baseline_pipeline),
    ("LinearSVC", svc_pipeline),
]:
    scores = cross_val_score(pipeline, X, y, cv=cv, scoring='recall_macro', n_jobs=-1)
    print(f" {name:<25} Mean={scores.mean():.4f} Std={scores.std():.4f} {scores.rou

print()

=== 5-FOLD STRATIFIED CROSS-VALIDATION (Macro Recall) ===

Logistic Regression      Mean=0.7743 Std=0.0145 [0.769, 0.76, 0.795, 0.788, 0.76]
LinearSVC                Mean=0.7566 Std=0.0116 [0.755, 0.744, 0.767, 0.773, 0.74
4]
```

7. DEPLOYMENT

"Package the model for real-world use and deliver actionable recommendations to the stakeholders."

7.1 Live Prediction Demo

In [189...

```
def predict_sentiment(tweet_text, brand='unknown'):
    """
    Classify the sentiment of a new tweet using the best trained pipeline.

    Parameters
    -----
    tweet_text : str    Raw tweet text (will be cleaned automatically)
    brand      : str    'apple', 'google', or 'unknown'

    Returns
    -----
    label      : str    Predicted sentiment class
    confidence : dict    Per-class probabilities (Logistic Regression)
```

```

"""
input_df = pd.DataFrame({'clean_tweet': [clean_tweet(tweet_text)],
                             'target_brand': [brand]})
label = baseline_pipeline.predict(input_df)[0]
proba = baseline_pipeline.predict_proba(input_df)[0]
proba_dict = dict(zip(baseline_pipeline.classes_, proba.round(3)))
return label, proba_dict

# — Test on 8 new, unseen tweets —————
test_cases = [
    ("I absolutely love my new iPhone! Best phone I've ever had #Apple", "apple"),
    ("Google Maps keeps crashing constantly. This is so frustrating!", "google"),
    ("Just bought a new Android phone. Will see how it goes.", "google"),
    ("Apple's MacBook is way too expensive for what you get honestly", "apple"),
    ("Google Assistant helped me book a restaurant today. Really impressed!", "google"),
    ("My AirPods stopped working after 3 months. Apple quality is slipping", "apple"),
    ("Google Photos is incredible. Automatically organizes everything for free", "googl"),
    ("Thinking about getting an iPhone next year but not decided yet", "unknown"),
]

print("=" * 80)
print(f"{'TWEET':<52} {'PREDICTION':<35} CONF")
print("=" * 80)
for tweet, brand in test_cases:
    label, proba = predict_sentiment(tweet, brand)
    top_conf = max(proba.values())
    print(f"{'tweet':<51} {'label':<35} {'top_conf':.1%}")

```

```

=====
TWEET                                PREDICTION
CONF
=====
I absolutely love my new iPhone! Best phone I've ev  Positive emotion
70.5%
Google Maps keeps crashing constantly. This is so f  Positive emotion
49.7%
Just bought a new Android phone. Will see how it go  Positive emotion
55.3%
Apple's MacBook is way too expensive for what you g  Negative emotion
66.9%
Google Assistant helped me book a restaurant today.  Positive emotion
83.8%
My AirPods stopped working after 3 months. Apple qu  Negative emotion
57.5%
Google Photos is incredible. Automatically organize  Positive emotion
91.3%
Thinking about getting an iPhone next year but not   No emotion toward brand or product
88.8%

Thinking about getting an iPhone next year but not   No emotion toward brand or product
88.8%

```

7.2 Actionable Recommendations for the Stakeholders

In [190...

```

print("=" * 68)
print("  RECOMMENDATIONS FOR THE STAKEHOLDERS")
print("=" * 68)

recommendations = [
    (
        "1. Automate Sentiment Monitoring",

```



```

        "Right now, someone has to manually read tweets to understand\n"
        "    how customers feel. Our model can do this automatically,\n"
        "    reading thousands of tweets per minute and labelling each\n"
        "    one as Positive, Negative, or Neutral – instantly."
    ),
    (
        "2. Get Alerts When Customers Are Unhappy",
        "Whenever the model detects a Negative tweet, it can trigger\n"
        "    an automatic alert to the relevant team. This means Apple\n"
        "    or Google can respond to complaints within hours instead\n"
        "    of days – before the problem goes viral."
    ),
    (
        "3. Track Sentiment Trends Week by Week",
        "By running the model continuously, the team gets a weekly\n"
        "    report showing whether sentiment is improving or declining.\n"
        "    For example: did a new product launch make customers happier\n"
        "    or trigger more complaints?"
    ),
    (
        "4. Understand Which Products Need Attention",
        "The model can be extended to filter by product – iPhone,\n"
        "    MacBook, Pixel, Android – so product managers know exactly\n"
        "    which product is generating the most negative feedback\n"
        "    and where to focus their resources."
    ),
    (
        "5. Use Word Insights to Guide Marketing",
        "The model reveals which specific words appear most in negative\n"
        "    tweets – for example 'battery', 'price', 'crashing'.  

        "    Marketing teams can use this to directly address customer  

        "    pain points in their messaging and campaigns."
    ),
]

for title, detail in recommendations:
    print(f"\n >> {title}")
    print(f"    {detail}")

print()
print("=" * 68)
print("  BOTTOM LINE")
print("=" * 68)
print("""
    Instead of guessing how customers feel, Apple and Google can
    now KNOW – in real time, at scale, across thousands of tweets
    every single day.

    The model is not perfect (it catches 6 in 10 negative tweets)
    but it is already far more efficient than any manual process,
    and it improves as more labelled data becomes available.
""")

```

```

=====
RECOMMENDATIONS FOR THE STAKEHOLDERS
=====

```

```

>> 1. Automate Sentiment Monitoring
    Right now, someone has to manually read tweets to understand
    how customers feel. Our model can do this automatically,
    reading thousands of tweets per minute and labelling each

```

one as Positive, Negative, or Neutral – instantly.

>> 2. Get Alerts When Customers Are Unhappy

Whenever the model detects a Negative tweet, it can trigger an automatic alert to the relevant team. This means Apple or Google can respond to complaints within hours instead of days – before the problem goes viral.

>> 3. Track Sentiment Trends Week by Week

By running the model continuously, the team gets a weekly report showing whether sentiment is improving or declining. For example: did a new product launch make customers happier or trigger more complaints?

>> 4. Understand Which Products Need Attention

The model can be extended to filter by product – iPhone, MacBook, Pixel, Android – so product managers know exactly which product is generating the most negative feedback and where to focus their resources.

>> 5. Use Word Insights to Guide Marketing

The model reveals which specific words appear most in negative tweets – for example 'battery', 'price', 'crashing'. Marketing teams can use this to directly address customer pain points in their messaging and campaigns.

=====

BOTTOM LINE

=====

Instead of guessing how customers feel, Apple and Google can now KNOW – in real time, at scale, across thousands of tweets every single day.

The model is not perfect (it catches 6 in 10 negative tweets) but it is already far more efficient than any manual process, and it improves as more labelled data becomes available.